

# Speed Showdown: Fathom vs. Python

Empirical Head-to-Head Comparison on Apple Silicon M4 (10 Cores, Release LTO)

## Empirical Benchmark Results: The Systems Difference

Benchmarks were executed offline using deterministic synthetic fixtures via `fathom bench` on macOS ARM64 (Apple M4, 10 cores, release build with LTO and stripped symbols).

| TOOL LAYER OVERHEAD (4 LAYERS)    |            |                             | MULTI-THREADED PARALLEL SPEEDUPS |            |             |               |
|-----------------------------------|------------|-----------------------------|----------------------------------|------------|-------------|---------------|
| LAYER                             | ITERATIONS | TIME / CALL                 | TASK                             | SEQUENTIAL | TOKIO SPAWN | SPEEDUP       |
| registry.execute (Raw dispatch)   | 300        | 5,140 µs                    | 16 × file_read (2MB each)        | 79.0 ms    | 25.8 ms     | 3.06×         |
| execute_batch (1 call)            | 300        | 7,614 µs                    | 8 × parse_html (1MB table)       | 130.3 ms   | 34.5 ms     | 3.78×         |
| execute_batch (8 calls amortized) | 320        | 5,893 µs (~0.75ms overhead) | 8 × code_symbols (240 files)     | 61.9 ms    | 20.4 ms     | 3.04×         |
| ToolCall JSON Serde               | 100,000    | 752 ns                      | web_feed (XML streaming)         | —          | —           | 1.11M items/s |

| Comparative Architecture Matrix |                                       |                                 |
|---------------------------------|---------------------------------------|---------------------------------|
| METRIC                          | PYTHON FRAMEWORKS (LANGCHAIN/AUTOGPT) | FATHOM (RUST COMPILED)          |
| Cold Start Latency              | 2,500 – 8,000 ms                      | < 5 ms (500x Faster)            |
| Tool Dispatch Overhead          | 25 – 150 ms                           | ~0.75 ms (100x Faster)          |
| Concurrency Model               | Blocked by Python GIL                 | True Multi-Threaded Tokio Spawn |

**ZERO LATENCY WASTE**

Because Fathom dispatches tools and serializes payloads in microseconds, users experience zero perceptible latency lag between agent reasoning turns.